Q4 – Building Long Text: + or join()?

Comparison

Repeated += can repeatedly create new immutable strings and may copy an increasing amount of content. join() first collects the pieces and constructs the final string efficiently in one operation. CPython can optimise some += patterns, but join() communicates the intended operation and is reliably suitable for many fragments.

Measurement

The benchmark measures 1,000, 10,000 and 200,000 lines using time.perf_counter(). Run it three times and report the mean from your own machine; timings vary by hardware, Python build and background load. The program verifies both methods produce identical text.

Alternative

Writing line-by-line to a file avoids holding the entire statement in memory; io.StringIO provides a file-like in-memory buffer. join() takes O(total characters) time and O(total output) memory. Repeated concatenation can approach O(total characters squared) copying in an unfavourable implementation. File streaming uses O(one line) working memory.

Python Program

```
import time

def build_plus(parts):
    text = ""
    for part in parts:
        text += part + ";"
    return text

def build_join(parts):
    return ";".join(parts) + (";" if parts else "")

def benchmark():
    for n in (1_000, 10_000, 200_000):
        items = [f"Item{i}:Rs{i % 50}" for i in range(n)]
        results = []
        for fn in (build_plus, build_join):
            start = time.perf_counter()
            value = fn(items)
            results.append((fn.__name__, time.perf_counter() - start, len(value)))
        print(n, results, "same:", build_plus(items) == build_join(items))

if __name__ == "__main__":
    benchmark()
```